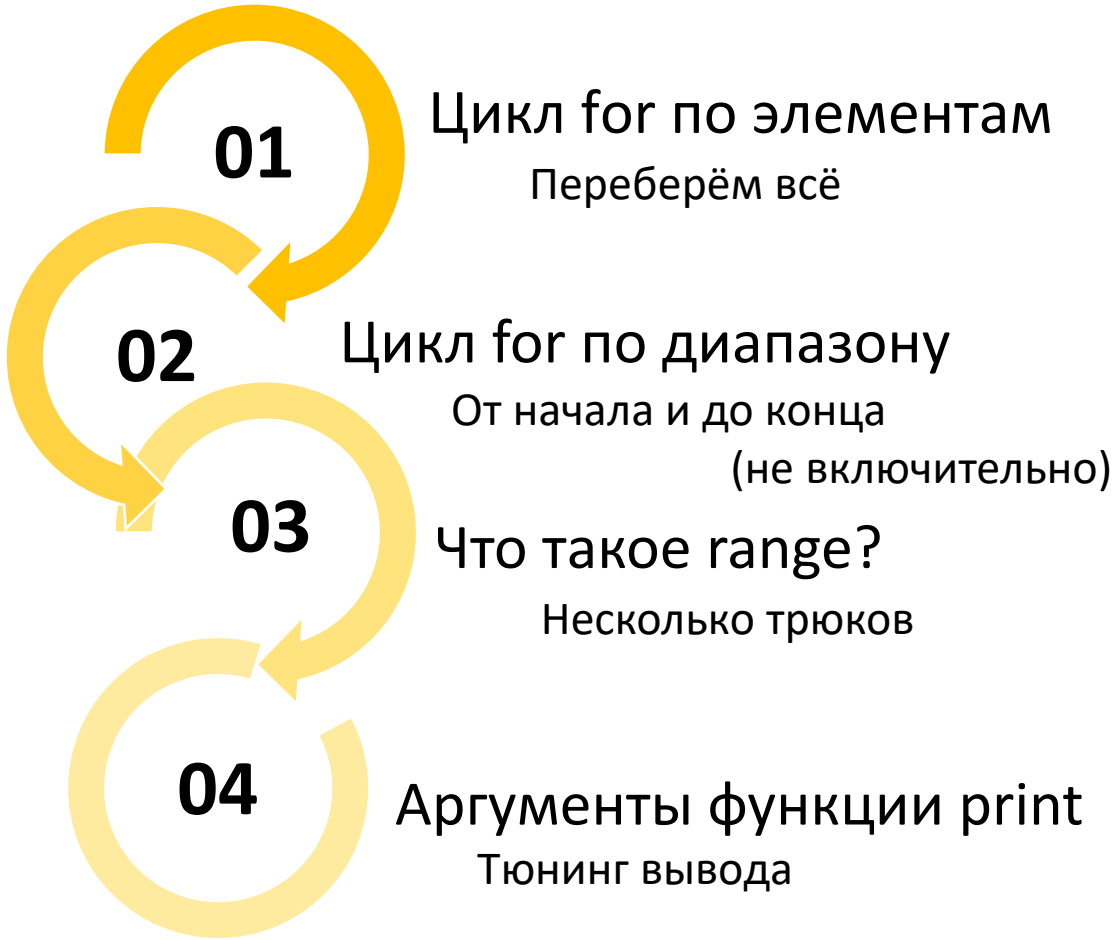

Яндекс

Яндекс Лицей

Оператор for

Не будем зацикливаться на while

Что впереди

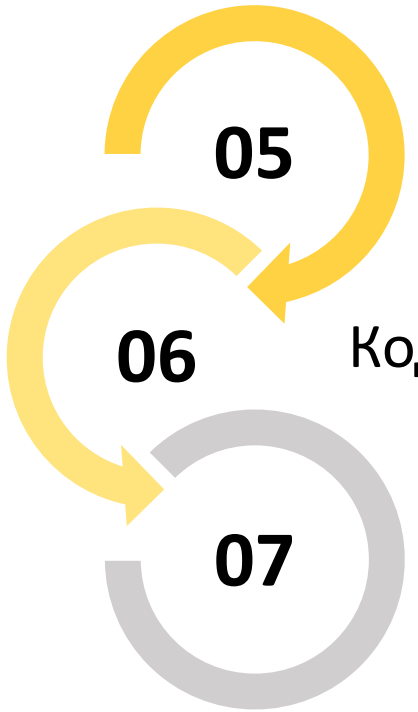


01 Цикл for по элементам
Переберём всё

02 Цикл for по диапазону
От начала и до конца
(не включительно)

03 Что такое range?
Несколько трюков

04 Аргументы функции print
Тюнинг вывода



05 Кодирование символов
Юникод в помощь

06 Коды букв и буквы по кодам
Уникальные коды

07 **Итоги**

Оператор for

Не будем зацикливаться на while

Цикл for по элементам

Представьте, что вы решили много раз написать одну и ту же фразу. Но делать одно и то же действие много раз подряд вручную — занятие скучное и бесполезное. К счастью, в Python есть специальный оператор цикла с известным числом повторений — for.

Задачу повторить какое-то действие n раз можно решить с помощью цикла while со счётчиком:

```
counter = 0
while counter < 100:
    print("Я выучу Python")
    counter += 1
```

```
for переменная in последовательность:
    # Делаем что-то с переменной.
```

```
for counter in числа от 0 до 100:
    # Делаем что-то с counter
```

Любой алгоритм, написанный с циклом while, можно переписать на for и наоборот.



FOR — Чёткий план

WHILE — Как получится

Задача цикла for в Python: **берём каждый элемент из последовательности и выполняем блок кода для этого элемента.**

Цикл for по элементам



Цикл for перебирает последовательности.

```
for num in 1, 2, 3, 4, 5:  # Выглядит как последовательность.
    print(num)  # Работает! Ну не чудо ли?
```

На первой итерации цикла переменная num принимает значение 1, затем 2 и так до конца.

Обратите внимание: до того, как начать выполнять тело цикла, интерпретатор **сначала вычислил все функции в заголовке** — заставил пользователя ввести три строки. То есть фактически код равнозначен этому:

```
for line in input(), input(), input():
    print('Пользователь ввёл строчку:')
    print(line)
```

```
line1 = input()
line2 = input()
line3 = input()
for line in line1, line2, line3:
    print('Пользователь ввёл строчку:')
    print(line)
```

В заголовке цикла можно перечислить то, что нужно перебирать. Тогда переменная цикла будет последовательно принимать заданные значения. А когда значения закончатся, то мы просто выйдем из цикла.

Цикл for по элементам

Помните, применительно к строкам мы несколько раз употребляли слово «коллекция» и даже искали в строке максимальную и минимальную букву? Может, можно перебирать буквы в строке?

```
for fruit in "🍎 🍌 🍊 ":
    print(f"Съел {fruit}.")  # Работает!
print("Фрукты закончились. Теперь чипсы?")
```

Более того, в заголовке цикла for работает и множественное присваивание. Пока посмотрим на следующий пример, как на волшебство, а уже скоро подробнее выясним, как оно там работает под капотом:

```
print('То, что вы скажете, будет '
      'использовано против вас побуквенно!')
line = input()
for letter in line:
    print(letter)
print('Как мы и предупреждали.')
```

```
for l1, l2 in 'он', 'ты', 'мы':
    print(f'В двухбуквенном слове {l1}{l2}')
    print(f'Первая буква {l1}, вторая - {l2}')
```

Будем последовательны



Давайте вернёмся к примеру с повторением строки 100 раз. В Python для решения такой задачи есть возможность создать последовательность целых чисел с помощью range():

```
for _ in range(100):
    print("Я выучу Python")
```

Получилось в два раза короче, чем с циклом while, и не надо менять счётчик самому.

Давайте внимательнее посмотрим, какую последовательность создаёт range().

```
for i in range(5):
    print(f"День {i}: Я всё ещё не умею летать 🐓")
```

День 0: Я всё ещё не умею летать 🐓
День 1: Я всё ещё не умею летать 🐓
День 2: Я всё ещё не умею летать 🐓
День 3: Я всё ещё не умею летать 🐓
День 4: Я всё ещё не умею летать 🐓

Цикл выполнился ровно пять раз, но i принимала значения 0, 1, 2, 3, 4. Это связано с тем, что в программировании принято начинать счёт с нуля. Именно поэтому range() выдаёт значения с нуля до верхней границы диапазона не включительно.

По соглашению **PEP8**, переменные цикла, которые используются как счётчики, можно называть одной буквой — i, j, k. Можно вообще использовать для имени нижнее подчёркивание _, но только в том случае, если мы хотим сделать акцент: внутри цикла переменная никак не используется и нужна лишь для организации нужного количества повторений.

Цикл for по диапазону



Хм, а что, если мы пишем программы не для других программистов, а для нормальных людей, которые считают с единицы? Есть два способа решения проблемы:

- Увеличивать значение на единицу непосредственно перед выводом:

```
print("Посчитаю от единицы до "
      "введённого числа (включительно)")
for i in range(int(input())):
    print(i + 1)
```

- Вызвать range() двумя параметрами — началом и концом диапазона:

```
print("Посчитаю от единицы до "
      "введённого числа (включительно)")
for i in range(1, int(input()) + 1):
    # Прибавили единицу, чтобы вошло
            само введённое число.
    print(i)
```

Значит, range() умеет принимать настройку не только конца диапазона (не включительно), но и начала (включительно).

Цикл for по диапазону



А ещё он может выдавать числа не подряд, а с некоторым шагом. Допустим, нам надо вывести все чётные числа от одного до введённого включительно. Мы можем написать if внутри цикла:

```
for i in range(1, int(input()) + 1):
    if i % 2 == 0:  # Перебираем все и ищем подходящее.
        print(f'{i} - чётное')
```

А можем задать шаг в качестве третьего параметра range():

```
for i in range(2, int(input()) + 1, 2):
    # Обратите внимание, что идём с двойки,
    # а то перебрали бы все нечётные числа.
    print(f'{i} - чётное')  # Выводим только нужное.
```

Никто не запрещает нам сделать шаг отрицательным. Тогда, чтобы цикл запустился хоть раз, надо задать start большим или равным stop:

```
# Считаем от 10 до 0 (не включительно) с шагом -1.
for countdown in range(10, 0, -1):
    print(f" 🚀  {countdown}...")
print("Ту зе мун!")
```

Цикл for по диапазону



Подытожим.
range() можно вызывать с разным количеством аргументов:

- С одним. range(stop) создаст последовательность чисел от нуля до stop не включительно.

```
for i in range(11):  # 0 1 2 3 4 5 6 7 8 9 10
```

- С двумя. range(start, stop) — последовательность от start включительно до stop не включительно.

```
for i in range(2, 12):  # 2 3 4 5 6 7 8 9 10 11
```

- С тремя. range(start, stop, step) — последовательность от start включительно до stop не включительно с шагом step.

```
for i in range(2, 23, 3):  # 2 5 8 11 14 17 20
```

Что такое этот range?

Пока важно знать, что range() не создаёт всю последовательность чисел сразу, а позволяет получать по одному числу за раз. Поэтому range(1_000_000_000_000_000) не съест всю оперативную память вашего устройства.

```
my_range = range(1, 100, 2)  # Можно сохранить в переменную.
```

Ещё несколько трюков с range():

```
for i in my_range:
    print(i)
```

```
for j in my_range:  # Начнём сначала.
    print(j)
```

```
print(my_range.start,  # Если вдруг забыли, можно узнать начало
      my_range.stop,  # конец
      my_range.step)  # и шаг
```

```
print(2 in my_range)  # узнать, входит ли какое-то значение в последовательность
```

```
print(len(my_range))  # узнать длину последовательности.
```





Аргументы функции print

Тюнингуем print

Аргументы функции print()

Функция print() в Python очень гибкая. У неё есть именованные аргументы (*keyword arguments*), позволяющие настроить разделитель между выводимыми частями и тем, что будет в конце вывода:

- sep (*separator*) — строка-разделитель, которая вставляется между аргументами при печати. По умолчанию sep=" " (пробел). Можно ставить любой разделитель: запятую, тире, смайл, секретное послание на 10 листах – что угодно, лишь бы это была строка.
- end — строка, которая добавляется в конце вывода. По умолчанию end="\n", то есть каждый print() заканчивается переводом строки. Сделаем так, чтобы два вызова print напечатали в одной строке:

```
print("Привет", end=", ")
print("пользователь", end="!\n")
# Вывод: Привет, пользователь! + перевод на следующую строку.
```

```
print("Cats", "Dogs", "Pythons", sep=" vs ")
# Вывод: Cats vs Dogs vs Pythons
```

PEP8

Вокруг знака = при указании значений именованных аргументов пробелы не нужны.

Оператор распаковки *

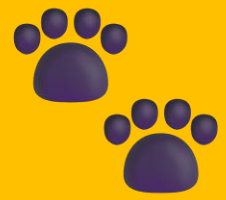
Мы будем использовать его могущество для того, чтобы print() воспринимал строки и range() не как единое целое, а как набор входящих внутрь букв или цифр.

```
sample_range = range(10)
print(sample_range)  # range(0, 10)
print(*sample_range)  # 0 1 2 3 4 5 6 7 8 9

sample_string = "превысокомногорассмотрительствующий"
print(sample_string)  # превысокомногорассмотрительствующий
print(*sample_string)  # п р е в ы с о к о м н о г о р а с с м о т р и т е л ь с т в у ю щ и й
```

А раз print() начинает воспринимать что-то не как одно значение, а как несколько, значит, с помощью параметра sep можно влиять на разделители между элементами.

```
print(*'Повторяю по буквам', sep='-')  # П-о-в-т-о-р-я-ю- -п-о- -б-у-к-в-а-м
print(*range(1, 101, 10), sep=", потом ")
# 1, потом 11, потом 21, потом 31, потом 41, потом 51, потом 61, потом 71, потом 81, потом 91
```



Почему цифровые котики

откликаются на последовательность
1050, 1080, 1089, 45, 1082, 1080, 1089, 33

Коды букв и буквы по кодам

Любой текст в памяти компьютера — это числа. А кодирование — это словарь перевода между буквами и числами.

В 1960-х годах инженеры думали: «Хватит ли 128 символов для всего мира?» Ответ: «Конечно! У нас же есть латиница, цифры и символы вроде @ и #». Так появилась ASCII — таблица, где каждому символу присваивался семибитный код (0-127).

Пример:

A → 65
a → 97
! → 33

Наконец, появилась идея создать единую таблицу для всех символов всех языков, включая эмодзи и древние руны. Так появился Unicode.

Пример:

```
print('\U0001F4A9') # 💩
```

Фишки Юникода:

- Каждому символу — уникальный код (например, А → U+0041, Б → U+0411, ☃ → U+2603).
- Поддерживает даже символы из вымышленных вселенных (например, клингонский).
- Коды не ограничены — таблица расширяется (сейчас > 150 000 символов).

Факт: В Python3 все строки — юникодные (тип str). Это значит:

```
text = 'Привет, 世界! 🌍' # Работает из коробки.
```

Коды букв и буквы по кодам

Иногда бывает нужно преобразовать символ в его код или наоборот, получить из кода символ, который ему соответствует.

- Функция ord() превращает символ в его юникод:

```
print(ord('A'))    # 65
print(ord('Б'))    # 1041
print(ord('☗'))    # 9731 (это 0x2603 в HEX)
```

- Функция chr() делает обратное — превращает код в символ:

```
print(chr(65))    # A
print(chr(1041))  # Б
print(chr(9731))  # ☃
```

Вот так можно вывести заглавный английский алфавит:

```
for i in range(26):
    print(chr(ord('A') + i), end=' ')
```

А так можно расшифровать послание из названия пункта учебника:

```
for i in 1050, 1080, 1089, 45, 1082, 1080, 1089, 33:
    print(chr(i), end='')
```

Так можно реализовать задачу шифра Цезаря: *при шифровании каждая буква исходного сообщения сдвигается на сколько-то позиций в одну сторону, а при расшифровке смещение происходит в обратную сторону.*




```
secret = ''
shift = int(input())
for char in 'HELLO':
    secret += chr(ord(char) + shift)
print(secret)  # Расшифровка: смещение назад на -shift.
```

Итог: цикл жизни нам стал чуть понятнее



Цикл for — это прекрасный помощник по автоматизации, который не пропустит ни одного элемента в списке дел (если мы его дополнительно не попросим).

А пока вы научились:

- создавать разные последовательности целых чисел с помощью range();
- перебирать числа, буквы и слова;
- совершили путешествие от букв к их кодам и обратно;
- поманили цифрового котика.

P. S. Если вы дочитали до конца — поздравляем, у вас успешно отработал цикл for по строкам этого урока.